

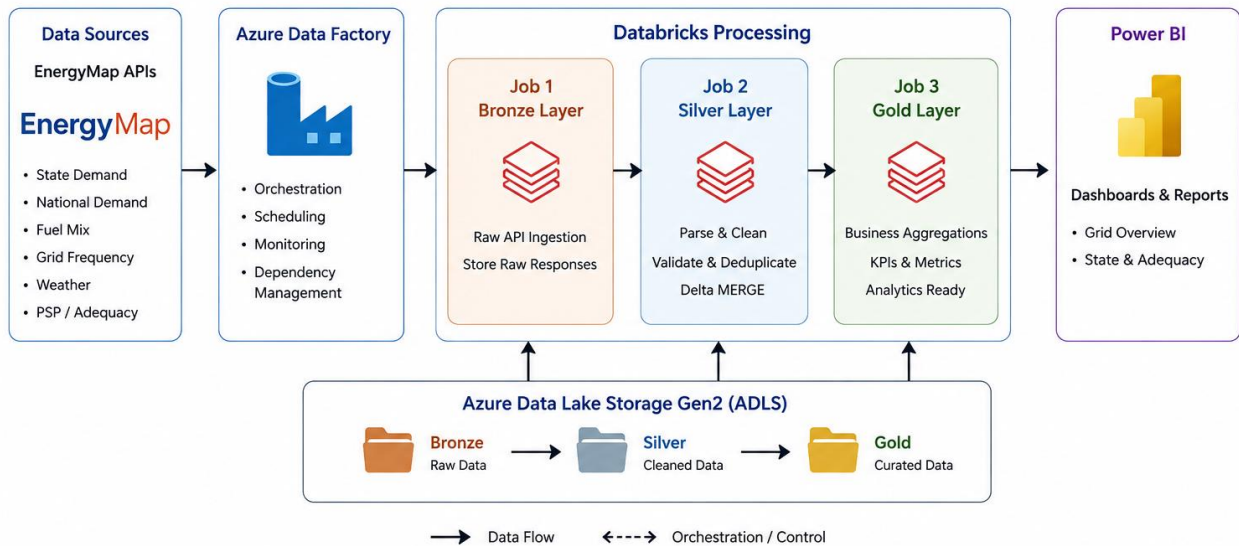
GridFlow Analytics

Technical Project Documentation

Personal Data Engineering & Electricity Analytics Project

GridFlow Analytics – Architecture

EnergyMap APIs → ADF → Databricks (Bronze → Silver → Gold) → Power BI



Final project architecture

Project focus	Electricity data engineering, incremental processing and analytics
Platform	Azure Data Factory, ADLS Gen2, Azure Databricks
Processing	Python, PySpark, Delta Lake
Analytics	Power BI
Delivery	GitHub + GitHub Actions

1. Project Overview

GridFlow Analytics is an end-to-end Azure Data Engineering and electricity analytics project. It ingests electricity, weather, generation-mix, grid-frequency, and adequacy data from EnergyMap APIs, processes the data through Bronze, Silver, and Gold layers following a Medallion Architecture, and presents curated analytics in Power BI.

2. Architecture

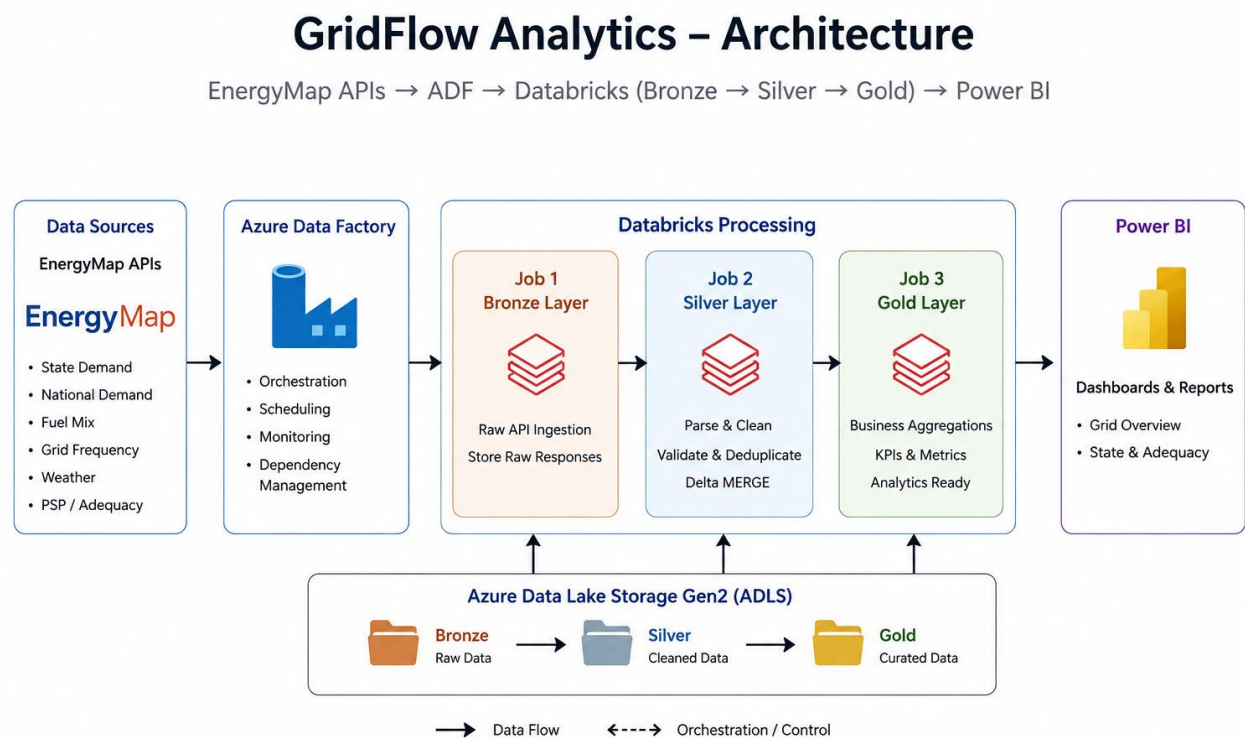


Figure 1. GridFlow Analytics architecture.

The main flow is EnergyMap APIs → Azure Data Factory → Databricks Bronze → Silver → Gold → Power BI. ADLS Gen2 stores the three data layers.

Component	Role
EnergyMap APIs	Electricity, weather, generation, frequency and adequacy source data.
Azure Data Factory	Orchestrates Bronze → Silver → Gold processing.
ADLS Gen2	Stores raw and processed datasets.
Databricks / PySpark	Parses, cleans, transforms and aggregates the data.
Delta Lake	Stores processed datasets and supports MERGE-based updates.
Power BI	Presents the final analytical views.

3. Data Pipeline

Bronze

Bronze preserves raw API responses with ingestion metadata. The code also protects against re-ingesting the same requested window for the supported ingestion datasets. PSP ingestion additionally hashes the complete raw response to avoid writing the same response twice.

Silver

Silver parses the `raw_response` field into dataset-specific canonical records, casts fields to appropriate types, removes invalid records where required, and applies dataset-specific deduplication and Delta MERGE keys.

Gold

Gold creates business-oriented datasets for national grid, state demand, state adequacy and state weather-demand analysis. Gold transformations also apply source selection where the business metric requires it, such as using official demand observations for the weather-demand analysis.

Incremental strategy

- Initial historical bootstrap followed by overlapping daily ingestion windows.
- Normal API ingestion uses a 48-hour window for the supported time-series datasets.
- Silver uses Bronze ingestion timestamps to identify new Bronze records.
- Gold processing uses the project's recent-window/incremental approach rather than rebuilding all historical data every run.

4. Azure Data Factory

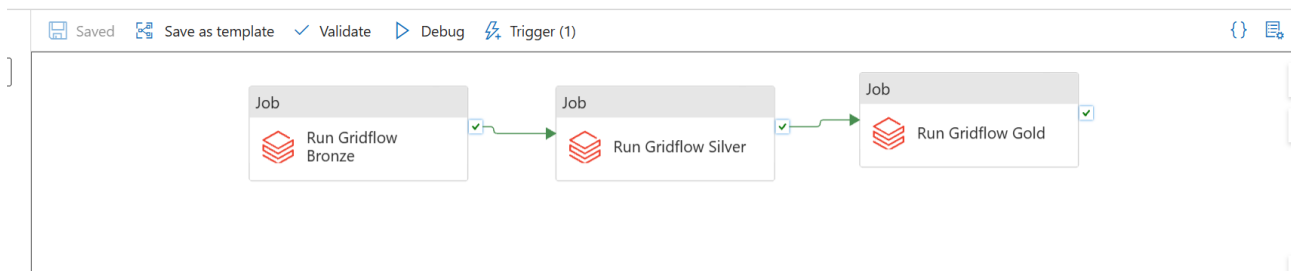


Figure 2. ADF orchestration pipeline.

ADF controls the execution order of the three main Databricks stages. Bronze must succeed before Silver, and Silver must succeed before Gold.

The current ADF activity configuration does not use automatic retries. A failed activity therefore stops downstream dependent stages and is recovered by rerunning the workflow.

Input validation

The ADF pipeline validates the supplied date range before starting processing. Invalid ranges fail early, while an omitted date range uses the pipeline's default recent window.

5. Databricks Processing

Bronze job

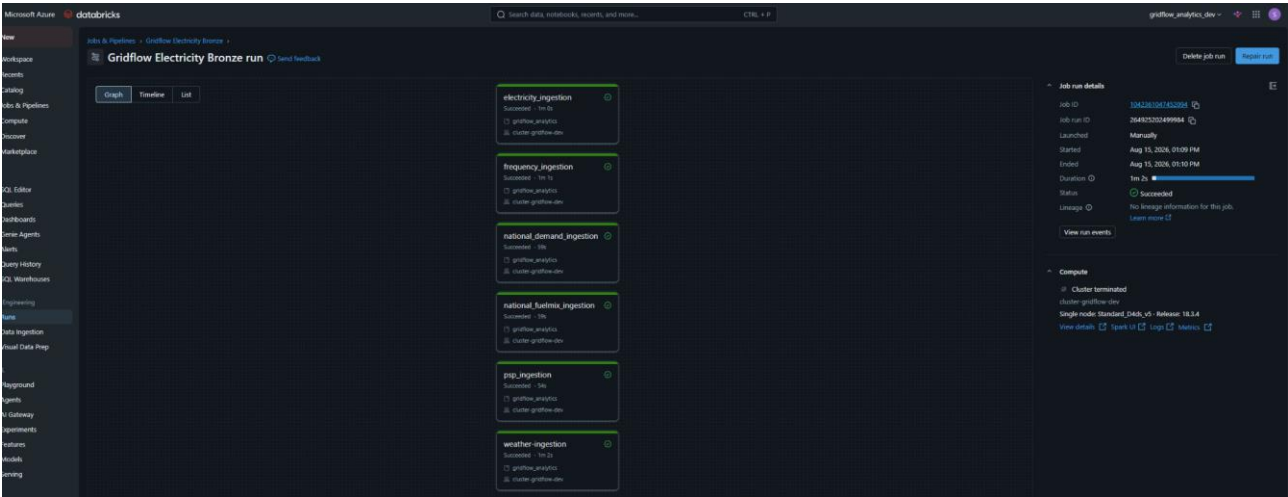


Figure 3. Bronze job run.

The Bronze job fetches the configured EnergyMap datasets and writes raw responses with ingestion metadata.

Silver job

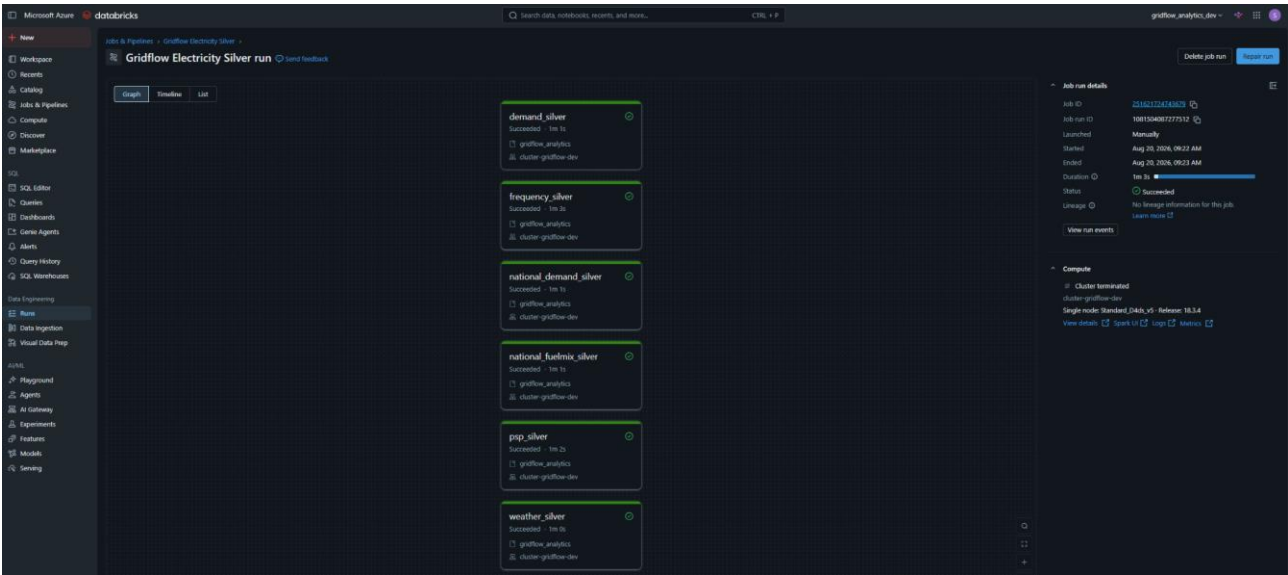


Figure 4. Silver job run.

The Silver jobs use a shared Bronze-reading utility that expects raw_response and infers the JSON structure from a sample response. Dataset-specific transformations then produce canonical records.

Gold job

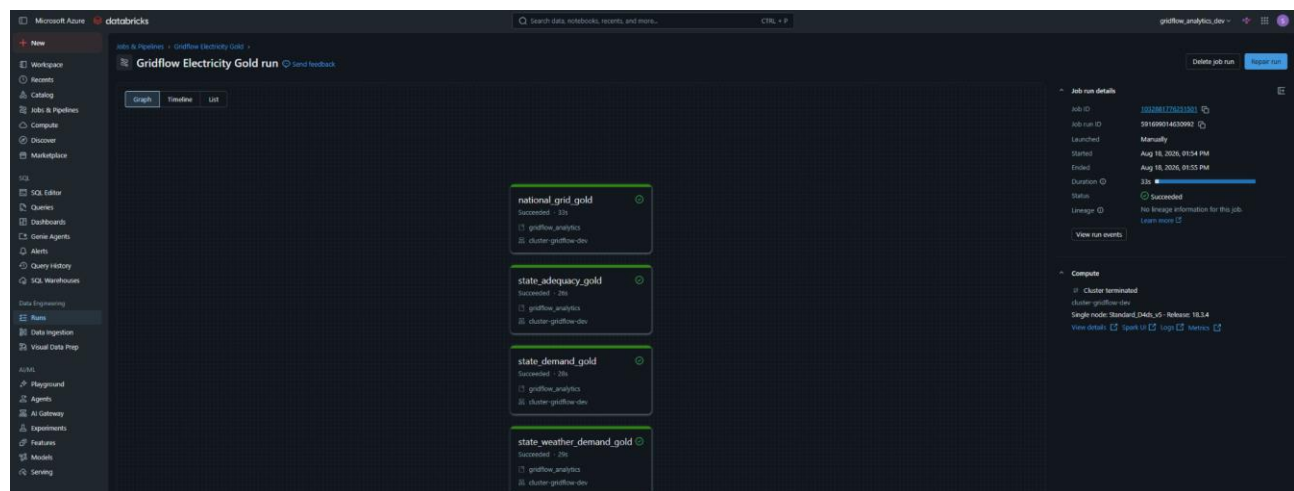


Figure 5. Gold job run.

The Gold jobs aggregate Silver data into dashboard-oriented datasets. The implementation is incremental, but the current code should be viewed as a practical portfolio implementation rather than a fully generalized production incremental-processing framework.

6. Data Quality, Duplicates and Provenance

- Duplicate ingestion is handled at Bronze for supported requested windows.
- Silver deduplication uses dataset-specific business keys rather than one universal key.
- For example, state demand uses state + timestamp + source; national fuel mix uses timestamp + fuel + source; PSP uses date + region + state + source_url.
- Modeled and official observations are retained as distinguishable source/provenance values.
- NULL values are not automatically discarded;
- State keys are normalized with trim/lower before the weather-demand join.
- Raw API responses are preserved so parser changes can be applied without losing the original source payload.

7. Handling Common Failure Scenarios

Scenario	Current GridFlow approach
Same API window ingested twice	Bronze checks existing ingestion-window metadata and skips an already ingested window where supported.
Same observation appears twice	Silver uses dataset-specific deduplication and Delta MERGE keys.
Two records have same timestamp	Do not assume duplicate; source/provenance can be part of the business key.
API request fails	Timeout/HTTP error is logged and raised; the task fails rather than silently writing incomplete data.
ADF stage fails	Downstream dependent stages do not run; the pipeline can be rerun.
Late-arriving data	Overlapping ingestion plus incremental/rolling recent processing helps incorporate recent source changes.
State names differ between sources	Normalize join keys before joining.
Secret/API key	Retrieved from Databricks Secrets at runtime rather than hard-coded.

8. Power BI

Grid Overview

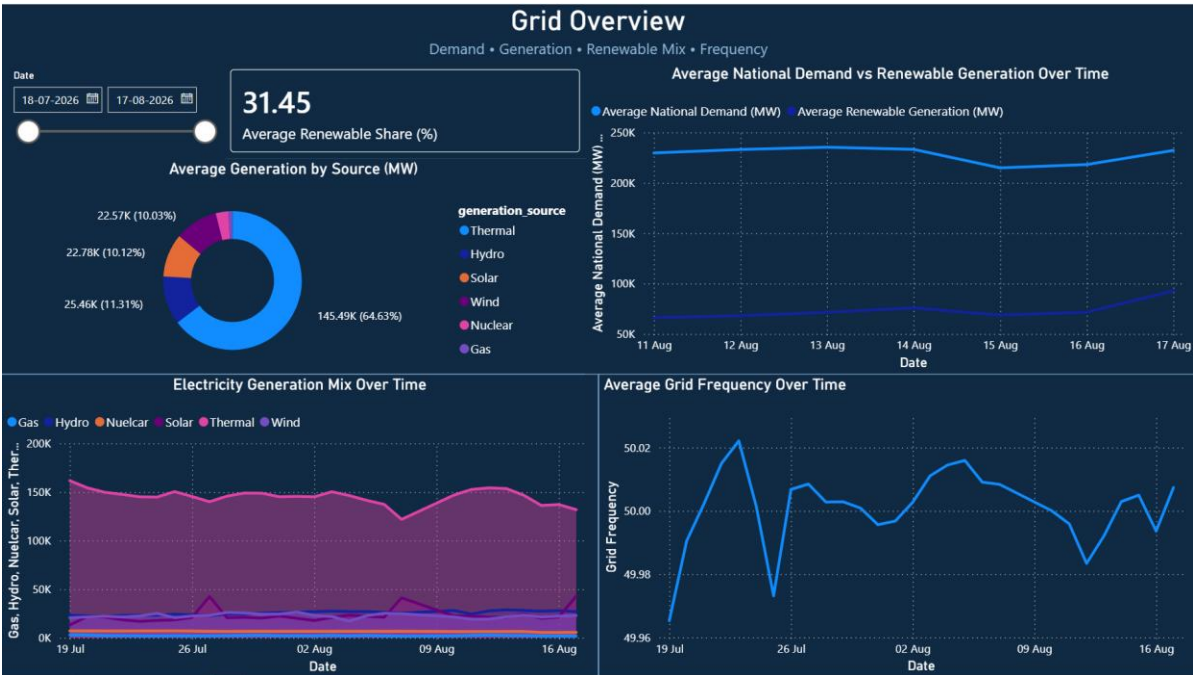


Figure 6. Grid Overview dashboard.

The Grid Overview page presents national demand, generation, renewable share, generation mix and frequency trends.

State & Adequacy

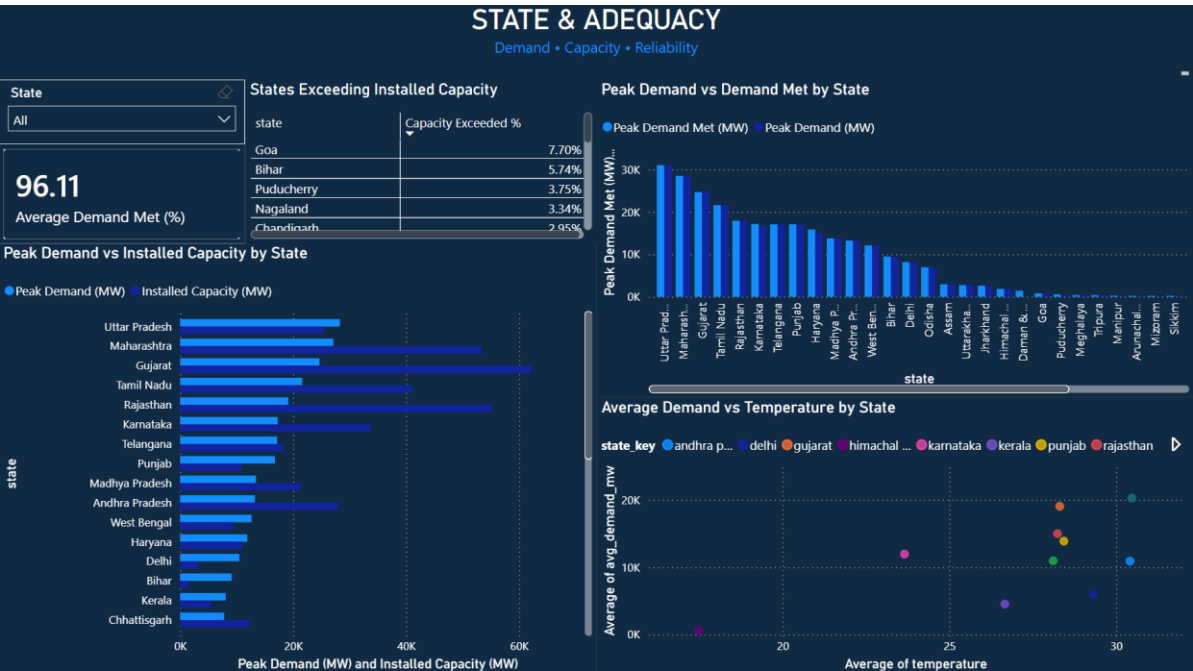


Figure 7. State & Adequacy dashboard.

The State & Adequacy page presents state demand, demand-met, peak demand, capacity and weather-demand analysis.

9. Python Packaging and CI/CD

The Python application is packaged as a Wheel. This avoids relying on workspace-relative source imports and gives Databricks a repeatable package artifact.

CI

- Compile Python source.
- Build the Wheel.
- Verify Wheel contents.
- Install the Wheel.
- Verify the installed distribution.

CD

CD is manually triggered. It builds the Wheel and uploads the artifact to the configured Databricks Volume using GitHub Actions secrets.

10. Key Engineering Decisions

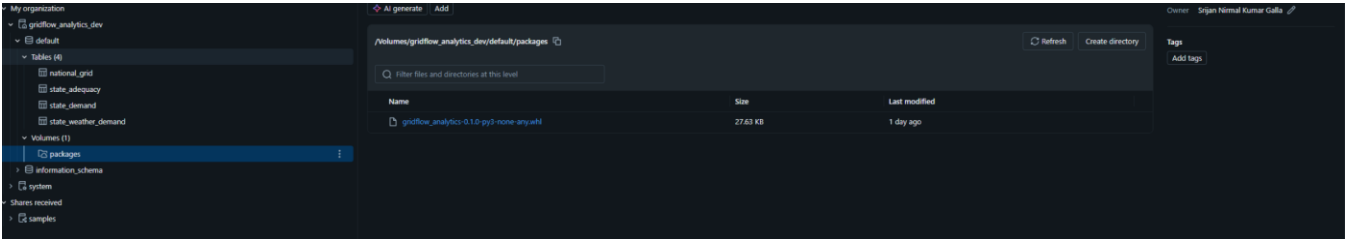
Decision	Why
ADF + Databricks	Keep orchestration separate from distributed Spark processing.
Bronze raw preservation	Allow inspection and reprocessing of the original API response.
Dataset-specific business keys	Avoid treating legitimate source variants as duplicates.
Delta MERGE	Handle overlapping observations without repeatedly appending identical analytical records.
48-hour overlap	Provide a simple recovery mechanism for missed daily ingestion.
Canonical Silver schemas	Keep Gold transformations independent of raw API structure.
Python Wheel	Provide a proper package artifact for Databricks and CI/CD.

Figure 8. Databricks catalog showing the project's curated tables and the deployed Python Wheel package in the project Volume.

As new Gold data accumulates, the Power BI dashboard uses the project's external Delta tables as its analytical source. This keeps the dashboard connected to the curated data produced by the pipeline rather than to manually exported files. The dashboard therefore reflects the continuing accumulation of pipeline output as the underlying tables are updated and the report is refreshed according to the configured setup.

The deployed project is intended to run as a daily data workflow rather than as a one-time historical load. The Azure Data Factory pipeline is configured in the Azure environment to run daily. Each run retrieves the configured recent source window, processes Bronze → Silver → Gold, and accumulates new observations in the Delta-based data layers.

11. Daily Operation and Dashboard Refresh



12. Current Limitations and Future Improvements

These are deliberately documented rather than hidden. They describe what the current personal project does not yet implement and what could be added if the project were expanded. The project was built within Azure free/trial resource constraints and Power BI Free, so the implementation was intentionally scoped around the services and capacity available for the project rather than enterprise paid-tier capabilities.

Area	Current project	future improvement
API resilience	Timeouts and visible failures; reruns provide recovery.	Add retry/backoff, rate-limit handling and stronger external-source monitoring.
ADF retries	Current activities do not configure automatic retries.	Add appropriate retry policies and alerting.
Monitoring	ADF/Databricks logs and run history are available.	Add operational metrics, alerts and data-quality monitoring.
Schema evolution	Raw payload is preserved, but semantic changes require code/schema updates.	Add schema detection and controlled compatibility checks.
Data-quality framework	Validation is implemented within processing code.	Add reusable data-quality checks and thresholds.

These improvements are not presented as missing requirements for the portfolio project. They are the next level of hardening that would be appropriate for a larger or business-critical deployment.

13. Project Questions & Answers

Q: What happens if the pipeline fails?

A: The failed step stops the next step from running. We can rerun the pipeline, and our overlapping 48-hour window helps recover recent data.

Q: What happens if the same data comes twice?

A: We check for already processed ingestion windows in Bronze and use deduplication and Delta MERGE in Silver.

Q: Why did you use Bronze, Silver and Gold?

A: Bronze keeps the raw data, Silver cleans and structures it, and Gold prepares the data for reporting and dashboards.

Q: Why use a 48-hour window?

A: The pipeline runs daily, so the overlap helps handle missed runs and recently changed data.

Q: What happens if the API is down?

A: The API request fails, the error is logged, and the Databricks job fails instead of continuing with incomplete data. We can run the pipeline again later.

Q: How do you handle duplicate records?

A: We use dataset-specific keys and Delta MERGE instead of treating every record with the same timestamp as a duplicate.

Q: Why did you use Databricks?

A: Databricks gives us Spark/PySpark for processing the datasets and Delta Lake for reliable storage and MERGE operations.

Q: Why did you use Azure Data Factory?

A: ADF handles the pipeline orchestration and controls the order of Bronze → Silver → Gold processing.

Q: Why did you use a Python Wheel?

A: It lets us package the Python code, push it to GitHub, and use CI/CD to build and deploy it to Databricks easily.

Q: How does the dashboard get updated?

A: The ADF pipeline runs daily, new data is processed into the Delta tables, and Power BI uses those curated tables as its data source.

14. Repository Structure

```
gridflow_analytics/  
├── .github/  
│   └── workflows/  
├── adf/  
├── architecture/  
├── dashboard/  
├── docs/  
├── src/  
│   └── gridflow_analytics/  
├── .gitignore  
├── pyproject.toml  
└── README.md
```